

w3L16 Policy Search:

Ways of solving bandit problem based on not value fn
Directly learn prob with which we have to select the arm

w3 lec16-2 Policy Search v2 -	25min
w3 lec17-2 REINFORCE (cont'd) -	26min
w3 lec18 Contextual Bandits v2 -	12min
w3 lec19 Full RL Introduction v2 -	36min
w3 lec20 Returns, Value functions and MDPs v2 -	44min

2.5hrs

*****EQUATIONS*****

Binary Bandits ($R_t = 0,1$)

If $R_t = 1$,

$$\pi_{t+1}(a_t) = \pi_t(a_t) + \alpha(1 - \pi_t(a_t))$$

$$\pi_{t+1}(a') = \pi_t(a')(1 - \alpha) ; \text{Here } a' \neq a_t$$

Second eq. $\pi_t(a') + \alpha(0 - \pi_t(a')) = \pi_{t+1}(a')(1 - \alpha)$

If $R_t = 0$,

$$\pi_{t+1}(a_t) = \pi_t(a_t)(1 - \beta)$$

$$\pi_{t+1}(a') = \pi_t(a') + \beta[1 - \pi_t(a')]$$

Case1: $\alpha = \beta, L_{R-P}$

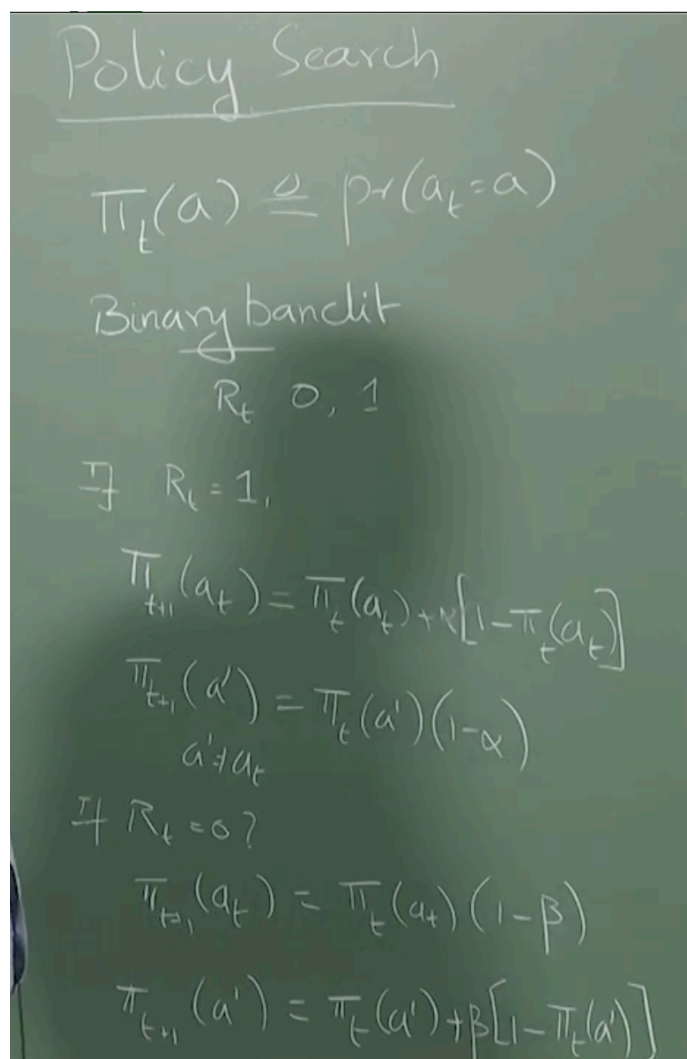
Case2: $\alpha \gg \beta, L_{R-\epsilon P}$

Case3: $\beta = 0, L_{R-I}$

1 in first equation nothing to do with reward, it is probability

For multiple action - take away alpha from every other action - sum will still be 1
 π_t will be 1 so π_{t+1} should also be 1

If I get a reward of 0, flip around.



Depending upon the relationship between alpha and beta are linear

1. When alpha = beta

- **LRP - Linear Reward Penalty:**
- These are there for many decades
- You change reward (1) and penalty (0)
- Change them by equal amount

2. When alpha >> beta (alpha is still small 10^{-3} , 10^{-4} , beta should 10^{-5} or 10^{-7})

- LR epsilon-p
- Much smaller change when I get penalty

3. When B -> 0 ->

- When rewards come i do something
- When penalty comes - don't do anything

Convergence is very different for all 3 algo

Best arm 0.6, bad 0.4, LRP will be good

Best .3, 0.25 LRP will have hard time

- Rewards are scars to come by, ignore penalty

Variable structure finite automata

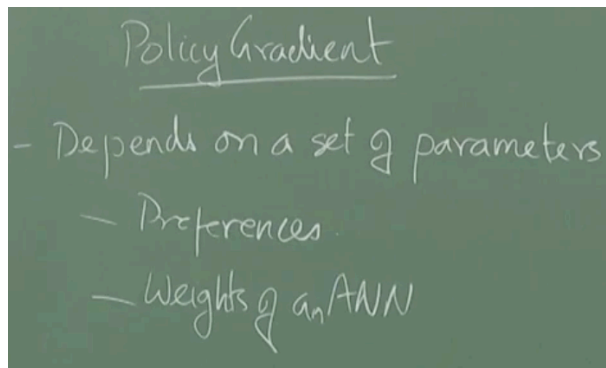
- Culmination of finite state machine
- Compact way of representing it

Policy representation - 1930 (prior to value functions approx)

Is there a more systematic way to learn policy directly

Policy Gradient approaches (book calls parameterized policy approaches)

- Assume policy depends on some set of parameters
- Policy is prob distr - specify it in many ways
 - E.g by a soft-max function, what is it that i need to give to compute
 - Need to tell exponent -> it need not be value, it can be something else
 - Action 1 put 10 in exponent, for action 2 put 20 (arbitrary numbers)
 - These are called -> preferences (is e-power thing)
 - Weights of neural network
 - There is some way to convert it to probability



Ref: Article Go-playing Agent - learns to play game of go

- No. of patterns are huge, rules are simple

How they formulated problem

- RL and ANN is hot thing - beat european champion by 5-0

$$\begin{aligned}
 \text{I} \quad R_t &= 1, \\
 \pi_{t+1}(a_t) &= \pi_t(a_t) + \alpha [1 - \pi_t(a_t)] \\
 \pi_{t+1}(a') &= \pi_t(a') (1 - \alpha) \\
 &\quad a' \neq a_t \\
 \text{II} \quad R_t &= 0? \\
 \pi_{t+1}(a_t) &= \pi_t(a_t) (1 - \beta) \\
 \pi_{t+1}(a') &= \pi_t(a') + \beta [1 - \pi_t(a')]
 \end{aligned}$$

Bernoulli (single trial), Binomial (2 out come multiple trials) - depending upon it is one trial or repeated trial

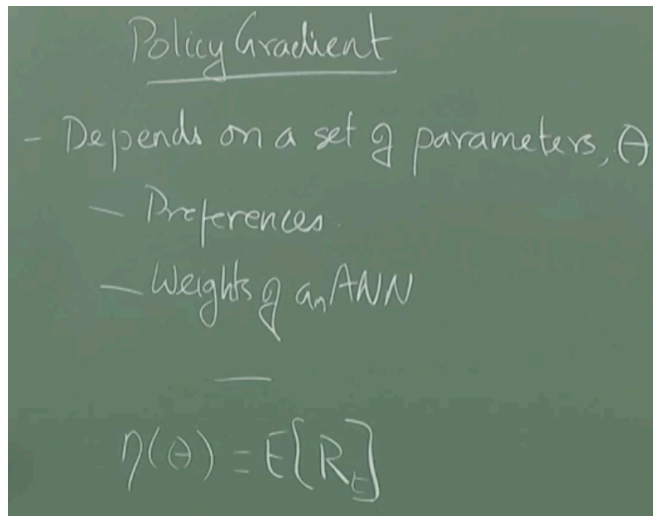
Binomial -> eq. Multinomial

Multinomial (multiple out come multiple experiment) - multiple outcome single experiment (categorical distribution)

What are the para describe Binomial distribution - prob of one

$$\begin{aligned}
 \pi_{t+1}(a_t) &= \pi_t(a_t) + \alpha [1 - \pi_t(a_t)] \\
 \pi_{t+1}(a') &= \pi_t(a') (1 - \alpha) \\
 &\quad a' \neq a_t
 \end{aligned}$$

w3L17 REINFORCE:



One policy - corresponds to $\rightarrow \theta_1, \theta_2, \dots, \theta_k$

$\eta(\theta)$ - i am evaluating that policy somehow

What is the most natural way - Expected payoff

$$\begin{aligned}\eta(\theta) &= E[R_t] \\ &= \sum_a Q_{\pi^*}(a) \cdot \pi(a, \theta)\end{aligned}$$

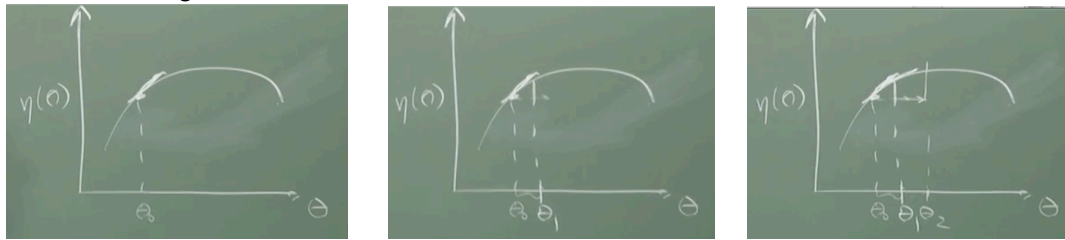
Std notation $\pi(\cdot; \theta)$ π is fn parameterized by θ - dot is input to function

Semi-colon - parameterised by θ $\pi(\cdot; \theta)$

Gradient Ascent: $\theta \leftarrow \theta + \alpha \nabla_{\theta} \eta(\theta)$

Add θ by increasing - gradient and θ

Performance goes like this



Why are we doing in incremental way

You don't know $q^*(a)$ function - doing sampling and finding gradient

- Gradient will not be correct - for 2 direction
- When there are multiple dimensions - you make small step
- The direction may be wrong for some and right for so take small steps

Stochastic gradient - don't know true gradient, you make estimate

SGDescent:

In an expected sense in many many time - you will move in right direction of gradient

how will you compute gradient if you don't know the function itself

$$\eta(\theta) = E[R_t]$$

$$= \sum_a q_{\theta^*}(a) \cdot \pi(a, \theta)$$

Gradient for this expression

$$\nabla_{\theta} \eta(\theta) = \sum_a q_{\theta^*}(a) \cdot \nabla_{\theta} \pi(a, \theta)$$

assuming q^* is parameter of problem - it is considered constant - something to do with problem

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \sum_a q_{\pi^*}(a) \cdot \nabla_{\theta} \pi(a; \theta) \\ &= \sum_a \left(q_{\pi^*}(a) \frac{\nabla_{\theta} \pi(a; \theta)}{\pi(a; \theta)} \right) \pi(a; \theta)\end{aligned}$$

multiply and divide gradient. π is nonzero for all a

Looks like expectation computation

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \sum_a q_{\pi^*}(a) \cdot \nabla_{\theta} \pi(a; \theta) \\ &= \sum_a \left(q_{\pi^*}(a) \frac{\nabla_{\theta} \pi(a; \theta)}{\pi(a; \theta)} \right) \pi(a; \theta) \\ &= E_{\pi(\cdot; \theta)} \left[q_{\pi^*}(a) \cdot \frac{\nabla_{\theta} \pi(a; \theta)}{\pi(a; \theta)} \right]\end{aligned}$$

Expectation is taken wrt to π

Draw sample and take average

q^* is expectation $\rightarrow$

Pull an arm (it is a sample), you get reward. Expectation of reward figures here

Expectation is taken over π and the process that generates reward

Can't use just R_t

Sample drawn is:

$$\begin{aligned}&= E_{\pi(\cdot; \theta)} \left[q_{\pi^*}(a) \cdot \frac{\nabla_{\theta} \pi(a; \theta)}{\pi(a; \theta)} \right] \\ &= R_t \cdot \frac{\nabla_{\theta} \pi(a; \theta)}{\pi(a; \theta)}\end{aligned}$$

$$= E_{\pi(\cdot|\theta)} \left[q_{\pi^*}(a) \cdot \frac{\nabla_{\theta} \pi(a, \theta)}{\pi(a, \theta)} \right]$$

$$\approx \frac{1}{N} \sum_{n=1}^N R_n \frac{\nabla_{\theta} \pi(a_n, \theta)}{\pi(a_n, \theta)}$$

n for discrete time, t for continuous time

This give gradient

Policy Search

$$\pi_t(a) \triangleq \Pr(a_t = a)$$

Binary bandit

$$R_t \in \{0, 1\}$$

If $R_t = 1$,

$$\pi_{t+1}(a_t) = \pi_t(a_t) + \alpha [1 - \pi_t(a_t)]$$

$$\pi_{t+1}(a') = \pi_t(a') (1 - \alpha) \quad a' \neq a_t$$

If $R_t = 0$?

$$\pi_{t+1}(a_t) = \pi_t(a_t) (1 - \beta)$$

$$\pi_{t+1}(a') = \pi_t(a') + \beta [1 - \pi_t(a')]$$

Sorry: for this part also erase t and write n

$$= E_{\pi(\cdot|\theta)} \left[q_{V^*}(a) \cdot \frac{\nabla_{\theta} \pi(a, \theta)}{\pi(a, \theta)} \right]$$

$$\approx \frac{1}{N} \sum_{\eta=1}^N R_{\eta} \frac{\nabla_{\theta} \pi(a_{\eta}, \theta)}{\pi(a_{\eta}, \theta)}$$

to compute 2nd term we know pi so take

derivative.

It could be Softmax, Continuation action (Gaussian)
Multinomial (derivative is simple)

Estimating q^* need not be right thing to do

q^* - turns out to be complex of the kth state

Direct representation of policy is lot more simpler

- Inventory - below certain level buy
- Value fn - how many items of different types are left

Policy gradient

- Generalise to continue action space

$$= E_{\pi(\cdot|\theta)} \left[q_{V^*}(a) \cdot \frac{\nabla_{\theta} \pi(a, \theta)}{\pi(a, \theta)} \right]$$

$$\approx \frac{1}{N} \sum_{\eta=1}^N R_{\eta} \frac{\nabla_{\theta} \pi(a_{\eta}, \theta)}{\pi(a_{\eta}, \theta)}$$

This is batch update

Fixing theta - pulling arms N times, take average, compute gradient, change para

22min:

Incremental Version Θ_{n+1}

$$\Delta \Theta_n = \alpha_n R_n \frac{\nabla \pi(a_n; \Theta_n)}{\pi(a_n; \Theta_n)}$$
$$\Delta \Theta_n = \alpha_n R_n \frac{\partial \ln \pi(a_n; \Theta_n)}{\partial \Theta}$$

Have theta, pull arm, compute gradient, change para
At every step change para by some fn delta-theta

$$\Theta \leftarrow \Theta + \alpha \nabla \eta(\Theta)$$

theta = theta + delta-theta

$$\Theta_{n+1} = \Theta_n + \Delta \Theta_n$$

$$\Delta \Theta_n = \alpha_n R_n \frac{\partial \ln \pi(a_n; \Theta_n)}{\partial \Theta}$$

Taken ln so life will be easier as you take e to power in softmax

$$\Delta \Theta_n = \alpha_n (R_n - b_n) \frac{\partial \ln \pi(a_n; \Theta_n)}{\partial \Theta}$$

b_n reinforcement baseline - b_n should not be a fn of action i take, then it becomes dependent on theta

If you are above baseline - good reward, otherwise baseline

Take average of all rewards obtained so far - as baseline

If reward is good -> will go in the direction of gradient

$$\Delta \Theta_n = \alpha_n R_n \frac{\partial \ln \pi(a_n; \Theta_n)}{\partial \Theta}$$

$$\Delta \Theta_n = \alpha_n (R_n - b_n) \frac{\partial \ln \pi(a_n; \Theta_n)}{\partial \Theta}$$

Reinforcement Baseline

Characteristic Eligibility

Characteristic Eligibility

Maintain q^* (estimate - critic) and π representation (actor)

Why do i use R_n - why can't i plug in q^* -> q^* reduces variance at the cost of adding significant bias

Why is it called Characteristic Eligibility?

- Which are the thetas most responsible for the change that is happening
- If there are 3-4 para, which ever one has highest gradient
- Which theta is more eligible to get update
- Hence called Characteristic Eligibility

Incremental update is called - REINFORCE algorithm (Williams 1988)

- In context of neural network
- Thetas are weight of ANN
- It looks like a crazy update
- In an expected sense, gradient will be in right direction
- If i had computed correct gradient - in same direction

Extremely slow as variance is very high

- Horrible part about paper
- Each letter stands for a word
- I refuse to memorise what it stands for

33min: Consider Binary Bandit - or bandit with 2 actions - can have arbitrary reward

Two Actions, arb. rewards

$$\pi(a, \theta) = \begin{cases} \theta & \text{if } a=1 \\ 1-\theta & \text{if } a=0 \end{cases}$$

These parameters I chose
Bernoulli - 2 actions, with prob chose

$$\frac{\partial \ln \pi}{\partial \theta} = \frac{a - \theta}{\theta(1-\theta)}$$

The update is above. For theta 1/theta for 1-theta -> -(1/a-theta)

Compact way of writing

A = 1, will get 1/theta, if A = 0, will get -(1/a-theta)

$$\alpha = \rho \theta(1-\theta)$$

choose $\rho = 0$

Chose alpha to be some rho times

As long as alpha is not dependent on actual reward, you converge,
NOT dependent on actual action or reward you receive

$$\Delta \theta_n = \rho \cdot (a - \theta) \cdot R_n$$

So delta theta

Interpretation:

- Take action 1, reward 1 get $\rho \cdot (1-\theta)$

$$\pi_{t+1}(a_t) = \pi_t(a_t) + \rho [1 - \pi_t(a_t)]$$

- Second term in $\pi_{t+1}(a_t) = \pi_t(a_t) + \rho [1 - \pi_t(a_t)]$ reward was 1
- If action 1, reward = 0, nothing

$$\Delta \theta_n = \rho \cdot (a - \theta) \cdot R_n$$

- If action of 0, got reward of 1 -> $-\theta \cdot R_n$

Second term

$$\pi_{t+1}(a_t) = \pi_t(a_t) (1 - \rho)$$

LRI is gradient following algorithm - REINFORCE algorithm

You can try to look at various choice for alpha and pi -> come up with update rule

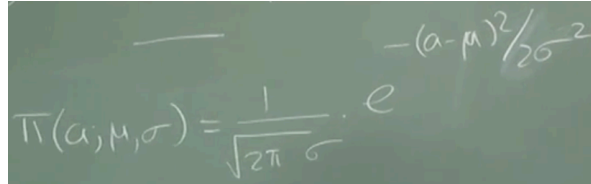
Homework 1:

Use softmax expression:

Replace all qs with arbitrary theta

Another Homework:

REINFORCE - allows to handle continuous actions



A photograph of a chalkboard with a handwritten Gaussian distribution formula. The formula is $\pi(a; \mu, \sigma) = \frac{1}{\sqrt{2\pi} \sigma} \cdot e^{-\frac{(a-\mu)^2}{2\sigma^2}}$. The text is written in white chalk on a dark green background.

- Will use

It is prob distribution - policy

Get me the REINFORCE rule

- One for mu
- One for sigma

Try to simplify as much as possible

Choose alpha so the constants get canceled out

- So you end up with nice looking form

w3L18 Contextual Bandits:

I am relaxing assumption that there are no state

From immediate RL -> Full RL

- Adding state dependence
- For diff state will have different reward
- We are missing sequential dependence
- Look at input and take action and move on
- I am not solving one problem but solving multiple bandit problems
- At each point what bandit problem we are solving
- Maintain - 10 diff sets of action values/ policies
 - Do independently and solve
- These problems can be many - huge
- Ad selection/ News story selection -
 - By treating bandit problem
 - Show one out of 20 stories - pull arm (reward 1 or 0)
- Everyone will get same news story - or when you go back
- Depending up on who come to your page
- Group people - for each group use different bandit
- How do i group them
 - Browsing history
 - Already have account -data filled in
 - Country
 - Phone laptop
- Want some kind of generalisation to do

Parameterisation of value, depending upon the state and action (action can have para)

- Arm -> news stories : Political article - which describe the
- Users -> state

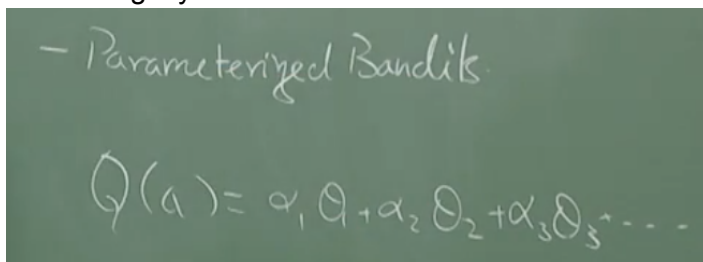
Parameterising - Value function

Prob setting - Contextual bandit

Way of solving: Linear bandit - value fn is assumed to be

Most deployment of bandit use it

- UCB -> most possible deployed version is Linear UCB
- $q^*(a)$ is some function of linear parameter
- Slightly diff rule then we do for UCB



- Parameterized Bandits.

$$Q(a) = \alpha_1 \theta_1 + \alpha_2 \theta_2 + \alpha_3 \theta_3 + \dots$$

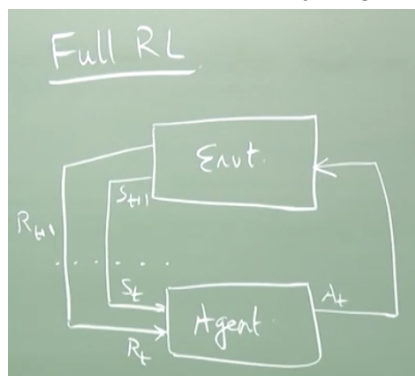
thetas will be para assume for representation

w3L19 Full RL Introduction:

Bandit = Immediate RL problem

Chapter 3 from book:

- Will insert sequential nature of the problem
- Assume there is an env, agent interacts with environment,
- At any point - agent sense state in which env is
- Agent takes action, which is applied to environment
- As a result, env changes to s_{t+1}
- Keep doing it - go on or we designated state or group of starting or stopping states
- Insert dummy state - one stop state and one start state
- Apart from that you get a reward signal from environment



Put rewards - outside the Agent's control

- Hence reward is put in env
- In reality it will be more complex
- Some mechanism in env which will give rewards

Where do rewards come from? 2009 Andrew Barto?

It is not even clear what is Agent and what is Environment

Robotics - reach and pick an object

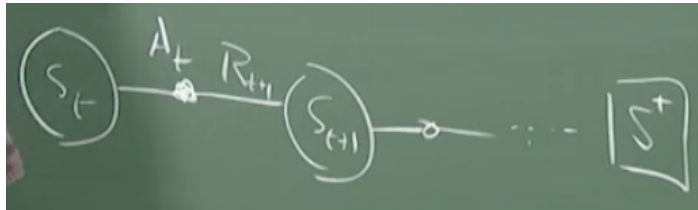
- Arm, angle of agent
- Controlling arm is agent
- Boundaries are not drawn neatly around reward
- Car driving - Agent could be car (I am controlling, env could be road (pedestrians))
- Env could be car - human is agent, where should i look at mirror, ahead
 - Human and car
- Can go further in human - which neuron in my cortex, nerve pushes the pedal
- I can draw boundary at different places

Don't expect people to come and give you agent and env

- You need to figure out the agent and env

Interactions:

- Next state and reward as determined at same time hence represented as $t+1$
- t could be discrete - most of the course it is discrete
 - t didn't even be time, it is step, **Decision Steps**,
 - point at which required to take decision
 - Decision need not happen at time t
 - Decisions may not take same time - chess - initially faster, later think and move
- With hierarchical RL will worry about duration of step
- S^+ -> terminal state, or indefinitely if there is no terminal state



Reward: Religiously follow chapter 3 of book

Classically - **rewards are scalar**, outside the direct control of agent

- Has to do through interaction with environment
- Might look weak reward - you can use scalar and get away with it
- Reaching task: how will you set rewards here
 - Once i grasp - give reward
 - Behave in such a way that you get reward in long run
 - Need to give some kind of reward
 - -ve reward for wrong move
 - Push animals
 - Is it sufficient, want to give additional pain if you do in someway
 - Reward of 10, with reward of -100 may not do
 - 99.1 is worse than 99.5 -> still getting -1s
 - Better form -> less -1 i get
 - -1000, will be more careful
- How do you set these reward functions
- Which are undesirable configurations
- Or get there as soon as i can - ok to crash/burn robots

May have to put multiple outcomes of same scale:

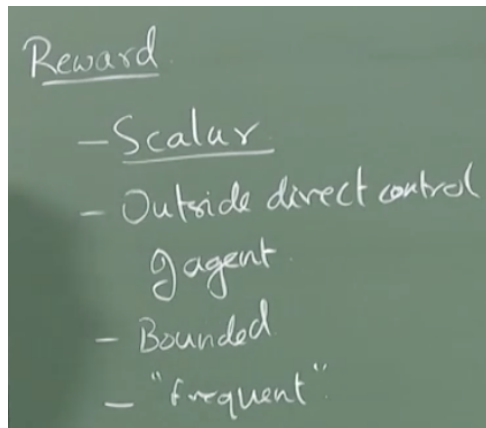
- Outcome with success, outcome with undesirable outcomes
- Instead of communicating 100 or -5

Benifits:

1. Scaler is powerful, gets in someway we do decision making / tradeoff
2. We put them on same scale
3. Having single value, can optimize outcomes

Mathematically we want rewards to be bounded

Rewards to be frequent (non zero) - not get once in a million time steps, you will not learn anything



Other type: multi-criteria RL - very few paper - multi-criterion

- **Multi-Agent setting**

Interesting to come up with Qualitative - Fuzzy - feedback is fuzzy signal

Policy:

$$\pi_t(a|s) \triangleq P_r(A_t=a | S_t=s)$$

- Prob that at time t - agent will take action a at time t the state was s

Stationary Policies:

$$\pi_1 = \pi_2 = \dots = \pi$$

at each time step it is same

$$P_r(A_1=a_1 | S_1=1) = P_r(A_{25}=a_1 | S_{25}=1)$$

state is 1, if $s_1=1$ or $s_{25}=1$, prob i take action a_2 will be same

Non-stationary:

- Collection of distribution set at each time step and state
- How you learn from data is different problem

Non-stationary - use π_t notation. For stationary use π notation

What is the Goal - i am trying to come up with the policy

- Learn something that will max reward in Long run
 - While riding cycling -> take out hand -> get injured
 - Short term feeling good
 - Chess - try to capture queen, end up losing game
- There are different time scales
- t index is interaction

REINFORCE

Policy Gradient Methods

- Policy depends on some parameters Θ
 - Action preferences
 - Mean and variance
 - Weights of a neural network
- Modify policy parameters directly instead of estimating the action values
- Maximize: $\eta(\Theta) = E(r)$

$$= \sum_a Q^*(a) \cdot \pi(\Theta, a)$$

$$\boxed{\Theta \leftarrow \Theta + \alpha \cdot \nabla \eta(\Theta)}$$

- Taking gradients:

$$\nabla \eta(\Theta) = \sum_a Q^*(a) \cdot \nabla \pi(\Theta, a)$$

- Rewriting:

$$\nabla \eta(\Theta) = \sum_a Q^*(a) \cdot \frac{\nabla \pi(\Theta, a)}{\pi(\Theta, a)} \pi(\Theta, a)$$

- Estimate gradient given N samples:

$$\hat{\nabla} \eta(\Theta) = \frac{1}{N} \sum_{t=1}^N r_t \cdot \frac{\nabla \pi(\Theta, a_t)}{\pi(\Theta, a_t)}$$

REINFORCE (Williams '92)

- Incremental version:

$$\Delta \theta_t = \alpha_t \cdot r_t \cdot \frac{\nabla \pi(\Theta, a_t)}{\pi(\Theta, a_t)}$$

Reinforcement Baseline

$$\Delta \theta_t = \alpha_t \cdot r_t \cdot \frac{\partial \ln \pi(\Theta, a_t)}{\partial \theta}$$

Characteristic Eligibility

$$\Delta \theta_t = \alpha_t \cdot (r_t - b_t) \cdot \frac{\partial \ln \pi(\Theta, a_t)}{\partial \theta}$$

Special case – Generalized L_{R-I}

- Consider binary bandit problems with arbitrary rewards

$$\pi(\theta, a) = \begin{cases} \theta & \text{if } a = 1 \\ 1 - \theta & \text{if } a = 0 \end{cases} \quad \frac{\partial \ln \pi}{\partial \theta} = \frac{a - \theta}{\theta(1 - \theta)}$$

$$b = 0 \text{ and } \alpha = \rho \cdot \theta(1 - \theta)$$

$$\Delta \theta = \rho \cdot r \cdot (a - \theta)$$

Reinforcement Comparison

- Set baseline to average of observed rewards

$$b_t = \bar{r}_t = \bar{r}_{t-1} + \beta \cdot (r_t - \bar{r}_{t-1})$$

- Softmax action selection

$$\Delta\theta_i = \alpha \cdot (r - \bar{r})(1 - \pi(\Theta, a_i))$$

$$\pi(\Theta, a_i) = \frac{e^{\theta_i}}{\sum_{j=1}^n e^{\theta_j}}$$

**Computation of
characteristic eligibility
for softmax action selection**

$$\begin{aligned} \frac{\partial \ln \pi(\Theta, a_i)}{\partial \theta_i} &= \frac{\partial}{\partial \theta_i} \ln \frac{e^{\theta_i}}{\sum_{j=1}^n e^{\theta_j}} \\ &= \frac{\partial}{\partial \theta_i} (\theta_i - \ln(\sum_{j=1}^n e^{\theta_j})) \\ &= 1 - \frac{e^{\theta_i}}{\sum_{j=1}^n e^{\theta_j}} \\ &= 1 - \pi(\Theta, a_i) \end{aligned}$$

Continuous Actions

- Use a Gaussian distribution to select actions

$$\pi(a, \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(a-\mu)^2}{2\sigma^2}}$$

- For suitable choice of parameters:

$$\Delta\mu = \alpha \cdot (r - \bar{r})(a - \mu)$$

$$\Delta\sigma = (\alpha / \sigma) \cdot (r - \bar{r})((a - \mu)^2 - \sigma^2)$$

w3L20 Returns, Value-functions and MDPs

Total Return: $G_t = R_{t+1} + R_{t+2} + \dots + R_T$

Long term - notion of Return G_T = end of episode - most natural way

- If T is finite, will take all rewards and optimise
- If I optimise only $t+1$ or $t+2$: may be put me bad in $s+2$
- Even if some of these no. is small, overall sum should be maximum

Issue one circumstance - when t is very large or infinite

- Will become quantity with variance

T is random variable -> every time I run system, it will stop after different move

- E.g chess -> not necessarily will end in same no. of moves
- It will be different values of T

G_T is also a random variable - as it is sum of random variables

So sum is bounded

Discounted Return $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots ; 0 \leq \gamma < 1$

Gamma = 0, optimising for immediate payoff

- Next time step what is the reward

Earlier Reward is more valuable than later Reward

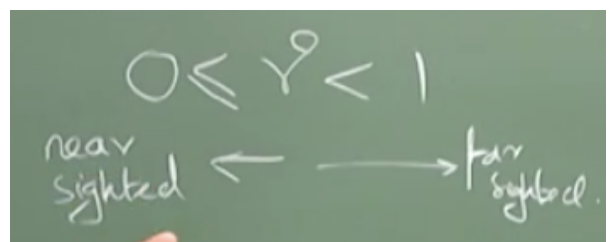
-1, -1 for no pickup, 100 for pickup

Gamma = 0.8 -> sooner i grab larger value I will get

- I will grasp it quickly
- Can get rid of -1
- What becomes tricky - what is the right value of gamma

Near sighted agent -> immediate gratification
Far sighted agent -> Higher payoff down the line

γ is 0.9999999. Then the bias is low



In one pull - i want to be optimum

RL people: - bandit as immediate RL Long term case - full RL	Bandit people: Interaction as entire learning process They consider it as an episode
---	---

Wherever I am - in last step I don't want to goof up

If T is given - only 4 more steps to go, only 3 steps to go

- It is non-stationary
- What to do

In chess - you may have some forecasting ability

- Basic setup T is unknown

Succeed for as long as you can, then die

RL (+ve and -ve rewards)

Psychology: rewards and punishments

Payoff, negative rewards (are called costs)

Average Reward Return:

$$G_t = \lim_{N \rightarrow \infty} \frac{1}{N} \{R_{t+1} + R_{t+2} + \dots + R_{t+N}\}$$

For finite no. of N it makes sense

For infinite N -> this might not have limit, will have to impose constraint - define limit

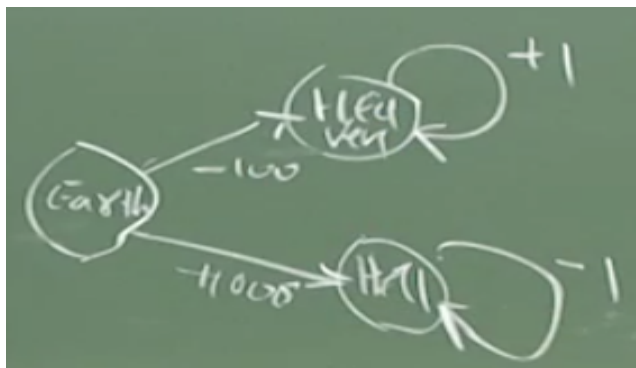
- If seq is periodic - limit won't exist
- How to handle limits - **cesaro's limit** ->
- take period as unit - let the no. of periods run to infinity

Doesn't have gamma parameter

- Nice from computation point of view

Gamma - allows to have personality to agent (near/far sighted)

Fun example:



gamma have to be high to go to heaven

Depending on gamma - policy will change

Average reward - going to heaven is better

- Will get infinite +1s for heaven
- Will get -infinity -1s for hell

Value fn:

Value Function:

Simplest Average of all rewards

True expected reward that i get q^* (q average maintaining)

$$G_t \triangleq R_{t+1} + R_{t+2} + \dots + R_T$$

Have policy - π - determines current reward and sequence I will see

V_π = value fn associated with policy π

$$V^\pi(s) = E_\pi\{G_t | S_t = s\}$$

v_π - value fn associated with

Take expectations of **Returns**

- In Bandit case - **rewards**
- Need to condition on π as G_t depends on π

$$G_t \triangleq R_{t+1} + R_{t+2} + \dots + R_T$$

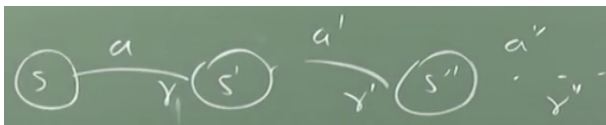
-

- Subsequent rewards depend actions i take, on π

$$V^\pi(s) = E_\pi\{G_t | S_t = s\}$$

-

- E_π = policy i am conditioning on
- Expectations wrt the distribution generated by following π of G_t
 - Given that at time t I started with state s
- Starting from state $s \rightarrow$ go to $s' \rightarrow s''$



-

on trajectory

- Have multiple trajectory from s
- Add them (or do discounted) - whatever average i get

I could pick different action, world is stochastic

Value function - summary:

1. π - is stochastic - take diff action

2. Take same a - but i get a different s' because of transition probabilities

Expectation is over all such sequences

$$q_{\pi}(s, a)$$

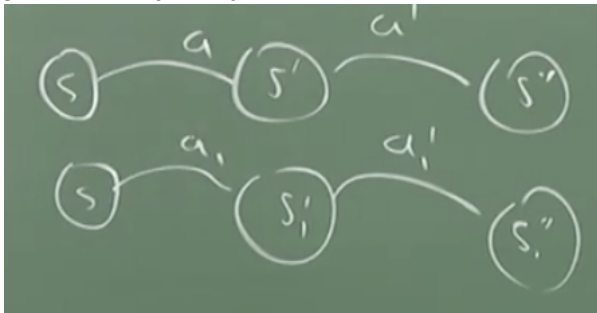
$q(a)$ - expected estimate for taking action a

Here taking action a in a specific state

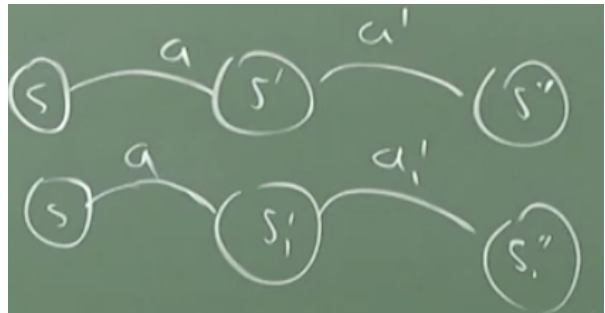
$$q_{\pi}(s, a) = E_{\pi} \{ G_t \mid S_t = s, A_t = a \}$$

What is the big difference between v and q

Value fn - pick diff actions, every time you generate trajectory



q function - first take action a in both cases
Then chose subsequent action



Subsequently pick action according to π

Gives information about: How good is it to perform action a in a given state

How do you define / characterize the problem?

In state s , take action, go to next state, and get reward

Need to tell you

- How transition happens and
- How reward generation happens

You want to know history, how you got there

$$Pr(S_{t+1}, R_{t+1} \mid S_t, A_t, S_{t-1}, A_{t-1} \dots S_0)$$

Prob of S_{t+1} and R_{t+1} what happened earlier

Make strong assumption - 1st Order Markov

- History doesn't matter
- Last state and action matter

$$Pr(S_{t+1}, R_{t+1} | S_t, A_t, S_{t-1}, A_{t-1} \dots S_0) = Pr(S_{t+1}, R_{t+1} | S_t, A_t)$$

History does not matter right only the current state and current action matters that will determine what the next state and next action will be right.

Stationarity assumption

$$Pr(S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a) = p(s', r | s, a)$$

Typically operate in Markov, they are stationary

To characterise problem what should i give you?

- The problem is defined by to define problem completely

$$S, A, p(s', r | s, a), \gamma$$

depends on what gamma i am using

- Depending upon gamma problem changes

In some cases, use simplified definition:

$$S, A, p(s' | s, a), E[r | s, a, s'], \gamma$$

This is how you describe a Markov Decision Process

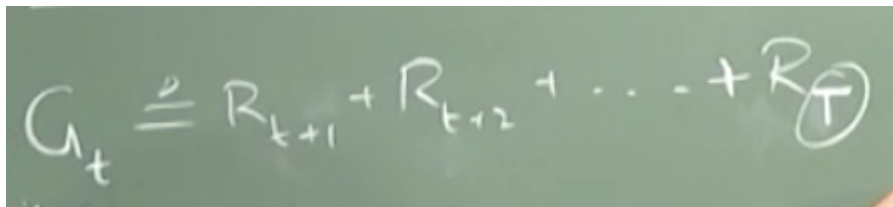
$p(s' | s, a)$ transition probability

For r conditioning on $r | s, a, s'$

Expected value of return - was in s , took action a , went to s')

- Giving the first moment of distribution

Why is it enough?


$$G_t \stackrel{p}{=} R_{t+1} + R_{t+2} + \dots + R_T$$

G_t composed of

Taking individual Expectations of each reward

$$(S, A, p(s'|s, a), E(r|s, a, s'), \gamma)$$

Markov Decision Process.

Evolution of states follow Markov process

We will assume:

- RL problems we are trying to solve can be modeled as MDP

For value function: Normally makes sense - when it is Markov

$$v^\pi(s) = E_\pi \left\{ G_t \mid s_t = s \right\}$$

I am assigning value to a state

- Not considering history - starting from state s

Marginalising over history

- Taken expectation over all history
- It works!

In non-Markov cases also will be using v-pi and q-pi

$$v^\pi(s) = E_\pi \left\{ G_t \mid s_t = s \right\}$$

$$q^\pi(s, a) = E_\pi \left\{ G_t \mid s_t = s, A_t = a \right\}$$